

Lista: Tipos Algébricos.

Nome: _____ Matrícula: _____

Exercícios baseados no capítulo 11 do livro Modern Programming Languages.

Exercício 01. Escreva uma definição de tipo datatype para um tipo naipes cujos valores são os quatro naipes de um baralho de cartas.

Exercício 02. Usando sua definição do Exercício 01, escreva uma função nome_naipes do tipo naipes -> string que retorna uma string dando o nome de um naipes.

Exercício 03. Escreva uma definição de tipo datatype para um tipo number cujos valores sejam inteiros ou números reais.

Exercício 04. Usando sua definição do Exercício 03, escreva uma função plus do tipo number -> number -> number que soma dois números, convertendo int para real somente se necessário.

Exercício 05. Escreva uma função addup do tipo intnest -> int que soma todos os inteiros em uma lista de intnest. Use esta definição para intnest:

`datatype intnest = INT of int | LIST of intnest list;`

(Tome cuidado ao digitar: INT não é o mesmo que int!)

Exercício 06. Escreva uma função prod do tipo int mylist -> int que recebe uma lista int mylist e retorna o produto de todos os elementos da lista. Se a lista for NIL, sua função deve retornar 1. Aqui está a definição de mylist:

`datatype 'el mylist = NIL | CONS of 'el * 'el mylist;`

Exercício 07. Escreva uma função reverse do tipo 'a mylist -> 'a mylist que recebe uma lista mylist e retorna uma lista mylist com todos os elementos em ordem inversa. (Use a definição de mylist do Exercício 06).

Exercício 08. Escreva uma função `append` do tipo `'a mylist -> 'a mylist -> 'a mylist` que recebe duas listas `mylist`, `a` e `b`, e retorna uma lista `mylist` contendo todos os elementos de `a` seguidos por todos os elementos de `b`. (Use a definição de `mylist` do Exercício 06).

Exercício 09. Escreva uma função `appendall` do tipo `'a list tree -> 'a list` que recebe uma árvore de listas e retorna o resultado de concatenar todas as listas. Coloque a lista de um nó nesta ordem: primeiro o conteúdo da subárvore esquerda, depois a lista no nó e, finalmente, o conteúdo da subárvore direita. Aqui está a definição de `tree`:

```
datatype 'data tree = Empty | Node of 'data tree * 'data *  
'data tree;
```

Exercício 10. Uma árvore binária completa é aquela em que cada `Node` tem dois filhos `Empty` ou dois filhos `Node`, mas não um de cada. Escreva uma função `complete` do tipo `'a tree -> bool` que verifica se uma árvore é completa. (Use a definição de `tree` do Exercício 09).